Development and Characterization of Critical Libraries using Vector Instructions for AJIT Processor

M.Tech Project Stage 2 Report

Submitted in partial fulfillment of the requirements

for the degree of

Master of Technology

by

Gauri Patrikar

(Roll No. 203076002)

Under the guidance of

Prof. Madhav Desai

Department of Electrical Engineering
Indian Institute of Technology Bombay
October 2022

Acknowledgement

I express my gratitude to my guide Prof. Madhav Desai for providing me the opportunity to work on this topic.

Gauri Patrikar
Electrical Engineering
IIT Bombay

Abstract

Vector instructions are a class of instructions that enable parallel processing. Using these instructions multiple data sets can be evaluated in a single clock cycle, exploiting data level parallelism, and boosting performance. The AJIT Processor, developed at IIT Bombay is based on the 32 bit SPARC-V8 ISA. 64 bit vector and floating point instructions were added to the processor to enhance its performance. Using these instructions critical libraries such as string were developed. String functions are used in many applications such as plagiarism check etc. These libraries have potential for parallelism. Strings work on 32 bit data chunks. This creates an opportunity for vectorization, which has been exploited using the AJIT vector instructions. These libraries were characterized for performance and it was found that performance improvement of up to 91% was observed as compared to the non vectored string functions.

Contents

List of Figures **ii**

1 Introduction **1**

2 Vector Instructions in the AJIT Processor **3**
2.1 Classes of instrucions 3
2.2 Using the new instructions 4

3 SPARC ABI **5**
3.1 Registers 5
3.2 Stack frame 5
3.3 Argument Passing 6
3.4 Prologue and epilogue 7

4 Implementation of string libraries **8**
4.1 Implementation strategy 8
4.1.1 a64strcpy 9
4.1.2 a64strcat 10
4.1.3 a64strncpy 10
4.1.4 a64strcmp 10
4.1.5 a64strcasecmp 11
4.1.6 a64strlen 11

5 Performance characterization **12**

6 Verification of libraries **14**

7 Conclusion **15**

List of Figures

3.1 Stack Frame Organization 6

4.1 source and destination alignments 9
4.2 newly loaded doubleword 9
4.3 intermediate doubleword 9
4.4 shifting of l0-l1 to get required byte 10

5.1 String function performance comparison 12

Chapter 1

Introduction

Vector instructions are widely used in applications such as AI and DSP. These instructions, known as SIMD (single instruction multiple data) perform the same operation on multiple data sets in a single clock cycle. This enhances the performance of any application as less iterations are required to process the same data as compared to non vector instructions. Many applications such as Image processing, signal processing , data analytics and machine learning etc, require huge amount of data to be processed. Vector instructions are very useful as multiple data sets can be computed in parallel.

To harness this capability, vector instructions were added in the AJIT processor. The AJIT processor is based on the 32 bit SPARC V8 architecture. The AJIT ISA was extended to have 64 bit vector instructions. These instructions operate on two 32 bit registers at once to give twice the performance compared to the 32 bit instructions.

One of the most useful 64 bit instruction used in the string library is zbytedpos. This instrucion tells us if and where there is a zero byte in an 8 byte data set. Strings terminate in null byte and hence all functions use this instruction to find the null byte position. Other most used instructions are the arithmetic and logical instructions. We use the addd function to increment two address locations in the same instruction.

To use these new instructions, assembler, disassmbler and debugger were developed. These tools enable the usage of the new vector instructions in assembly language programming for the AJIT processor.

Using these vector instructions, string functions were developed for the AJIT processor.

String functions are widely used in applications such as text search, information retrieval, web development and natural language processing (NLP) etc. They are used to search for specific phrases in a large file, or to extract data from a URL to check if it's valid. In NLP, it is used for operations such as tokenization (splitting text into smaller parts such as sentences) etc. As they generally operate on large data sets, we exploit the parallelism by operating on multiple data sets at once. The average URL length is 76 characters, but it can go upto 1600 characters. Plagiarism checkers can compare words or sentences or even larger chunks of text.NLP too works on understanding sentences and paragraphs. A sentence is on an average of 75-100 words, a large enough data set that can be vectorized. Another interesting application is spam message detection. A message is a maximum of 160 characters, we need to search the message to find certain words that m=will identify the message as spam.

We can find different string library implementations such as libastring, a string library developed by arm and optimized for their processors. Other open source implementations such as rapidstring, which is written in ANSI C, can be found on github.

As given above, we can only use these instructions in assembly language. Hence. these libraries are written in assembly. The SPARC application binary interface (ABI) was used. The ABI defines the rules for calling convention. We use these rules while writing

the libraries to ensure smooth operation of the program.

Chapter 2 introduces the vector instructions in the AJIT processor, chapter 3 introduces the SPARC ABI, and chapter 4 focus on the libraries that were implemented using these instructions. CHapter 5 details the verification process and chapter 6 looks into some popular benchmarks that use these string functions, chapter 7 concludes the thesis.

Chapter 2

Vector Instructions in the AJIT Processor

The AJIT processor is based on the 32 bit SPARC V8 architecture. In a 32 bit architecture, data is processed in 32 bit pieces, i.e. registers are of 32 bit each. If we can process larger amounts of data at once, for e.g. 64 bit data, we could potentially enhance the performance of many applications. Hence, vector instructions were introduced in the AJIT processor. These instructions can work on two 32 bit registers at once, giving us almost twice the performance boost. When data chunks are smaller like a byte, we can get up to 8 times the performance benefit using these instructions. We will look at the newly added instructions and then how to use them.

2.1 Classes of instrucions

The new instructions can be categorized as follows -

- Integer unit extensions - 64 bit arithmetic-logic-shift instructions. These accept 64 bit input operands and the output is also 64 bits. These have operations such as ADD, SUB, MUL, OR AND, XOR, SRL, SLLD, SRA. It operates on a 64 bit data chunk at once. These are used in almost all functions as we save one instruction when we increment the address of the source and destination in a single instruction.

- SIMD - These instructions are vector instructions which work on two source registers (each a 64 bit register pair), and produce a 64-bit vector result. The vector elements can be 8-bit/16-bit/32-bit. These instruction are very useful while operating on smaller data types For eg, for 8 bit data, we can perform the operation on eight 8 bit data in one clock cycle, reducing the number of instructions required to process the same amount of data.

- SIMD (bytereduce) - These instructions are vector instructions which reduce a 64 bit source register to a destination using an associative operation, such as, add, or , and and xor. It can do an 8 bit or 16 bit reduce. This has a very useful instruction - zbytedpos, which tells us if and where there is a zero byte in the doubleword. In the string library this instruction is heavily used to find the null byte.

- Floating point vector - These are vector float operations which work on two single precision operand pairs or 4 half precision operands to produce two single precision or half precision results respectively. These are similar to SIMD, only that they operate on floating point data. They have similar usage to that of SIMD instructions,

helping boost the performance by operating on multiple sets of data in a single clock cycle.

- Half precision conversion instructions - These instructions allow conversion between IEEE half-precision numbers and IEEE single/double precision numbers and integers.

2.2 Using the new instructions

As mentioned, these instructions can only be used in assembly language programming. This is because developing a compiler was a much more complex task. To use these instructions, assembler, disassembler and debugger were developed. Buildroot was used to cross compile for SPARC V8. The cross compiled toolchain provides compiler, assembler, linker etc for the SPARC V8 ISA. The new instructions were then added to buildroot, to generate the new assembler. To help with debugging, a disassembler and debugger ave also been built for the new instructions in the AJIT processor. We now have all the tools required to use these new instructions and get the performance boost.

Chapter 3

SPARC ABI

SPARC ABI is a calling convention used when an assembly program needs to be called using a C program. It gives the protocol for argument passing, writing the prologue and epilogue of the assembly functions and so on. This ensures that the process of shifting between the C and the assembly programs is seamless.

The libraries have been written in assembly, simply because generating a compiler was a more complex task. These libraries can be called from a C program, and hence the SPARC ABI protocol is used. We will refer to the C program as the caller, as it is the one calling the assembly program, which in turn is referred to as the callee.

3.1 Registers

The SPARC architecture provides windowed integer registers (in, out, local), global integer registers and floating-point registers. At any time, there are 32 integer and 32 floating point registers visible to the program. The 32 floating point registers are shared between caller and callee. The 32 integer registers are arranged as

- Global - shared between caller and callee.
- Out - for outgoing arguments to subroutine
- Local - caller and callee have their own local registers
- In - for incoming arguments to the subroutine

When a register window shifts, which occurs on a SAVE instruction, the callee function gets its own local and out registers, but the outs of the previous window become the ins of the new register window, i.e., the two windows share 8 registers. This is very useful for parameter passing, as outgoing parameters are stored in the out registers of the caller which the callee will find its in registers. The RESTORE instruction will restore the callers register window. The caller will now find the outputs of the program in its out register. Hence, the first 6 arguments can be passed using registers itself. Only 6 arguments can be passed as the last two registers of the out of the caller, o6 and o7, store the stack pointer and address of the call instruction respectively.

While using global registers, we need to note that g0 is always 0 and writes to it have no effect. g5-g7 are also reserved and cannot be used.

3.2 Stack frame

Every function also gets its own stack space. We allocate stack space to the callee at the start of the program itself with the SAVE instruction. The stack pointer points to

the bottom most address of the current stack and frame pointer points to the topmost address of the current stack. After a call, the current frame pointer of the callee is the stack pointer of the caller.

The figure given below shows the stack frame which is allocated to any new callee at compile time.

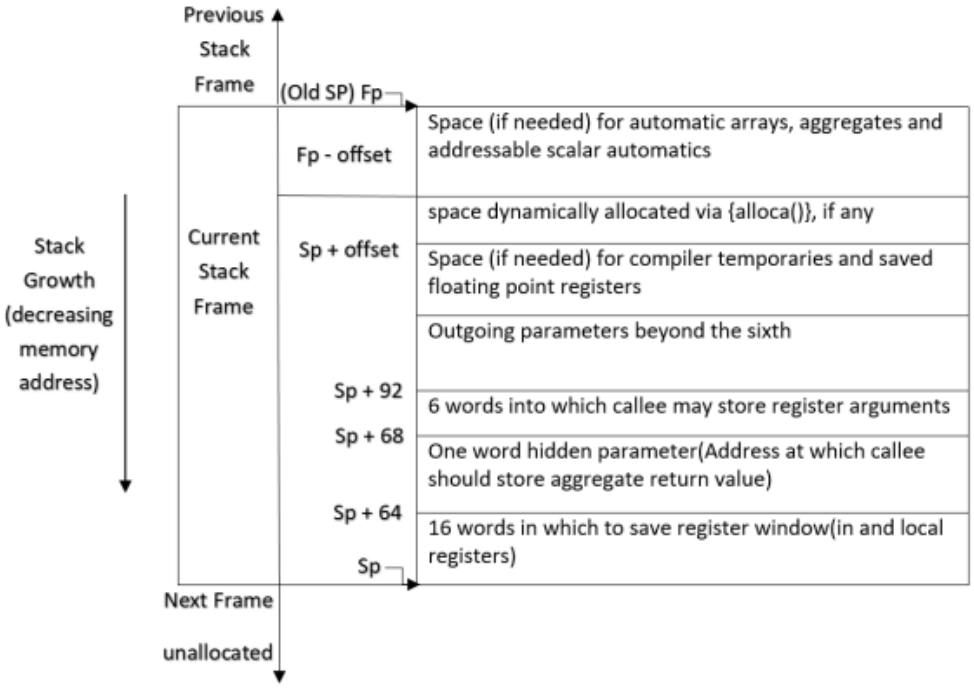


Figure 3.1: Stack Frame Organization

Addressable automatic variables on the stack are addressed with negative offsets relative to fp, everything else in the stack frame is addressed with positive offsets relative to sp. Incoming arguments are stored in the previous stack frame, which we will reference from positive offsets of frame pointer.

As only 6 arguments can be passed through registers, further arguments are passed using the stack.

3.3 Argument Passing

The first six integer input arguments are received through the in registers of the callee, and the rest of the arguments are passed through the stack. Functions pass all integer-valued arguments as words, expanding signed or unsigned bytes and halfwords as needed. The output is kept in the i0 register of the callee, the caller finds the value in o0. The in registers also hold the input floating point arguments, single precision need one register and double precision need two registers. As the floating point operations cannot use integer registers, compilers usually store them in the stack. A floating point return value appears in the floating point registers for both the caller and the callee. Single precision values appear in f0, while double precision in f0 and f1.

3.4 Prologue and epilogue

Each function that has to be called has a set epilogue and prologue. This ensures that new register window has been allocated when callee is called and that the previous window is restored when returning to the caller, to avoid any error in the program. It also take cares of moving between stack frames for the functions for smooth operation.

The prologue is as follows-

```
SAVE %sp, -96, %sp
```

Here, with the SAVE instruction, we are shifting to a new register window, making the out of the caller, the in of the callee, and giving the callee its own register window. Also we are allocating stack space to the callee. The stack size can be variable. The epilogue is as follows-

```
RESTORE
RETL
NOP
```

Here, the RESTORE instruction restores the previous register window of the callee. And the RETL instruction returns control to the caller.

Chapter 4

Implementation of string libraries

We will now take a closer look at the string library developed. Six functions of string library were implemented as listed below -

1. **a64strcpy** - equivalent to strcpy of the standard C library.
2. **a64strncpy** - equivalent to strncpy of the standard C library.
3. **a64strcmp** - equivalent to strcmp of the standard C library.
4. **a64strcasecmp** - equivalent to strcasecmp of the standard C library.
5. **a64strcat** - equivalent to strcat of the standard C library.
6. **a64strlen** - equivalent to strlen of the standard C library.

4.1 Implementation strategy

We will now see the details of the algorithm used to implement the libraries. The main challenge was that of making sure that the program works for any alignment of source and destination. The strings could be aligned to doubleword,word,halfword or byte boundary. In a doubleword as there are 8 bytes, 7 possible cases of alignment for each string need to be considered, combining to total of 49 possibilities. The main strategy that was used can be summarized as follows -

- If both are unaligned, first align the source to 8 byte boundary by performing byte wise operations. Now only the destination is unaligned.
- We then load a new doubleword from aligned source, and store/compare byte by byte with the destination till it has aligned to the doubleword boundary.
- We shift the remainder of the source string to another register pair, say r1. Then load a new doubleword from the source, and use it to fill the remaining part of register pair r1. Now you can store/compare it with the destination in one cycle, as destination will also be aligned.

Hence we can perform any operation with any alignment of source or destination string.

This strategy implementation requires addition, oring and shifting as a part of it. All these instructions are part of the 64 bit extension, hence all the string functions work on 64 bit data in a single iteration, which is the main factor in enhancing performance.

We will take a detailed look at the program flow for each function using an example. A step by step flow using an example for the function a64strcpy is given below. As all functions follow a similar flow, only the differences with respect to a64strcpy are mentioned for the remaining functions.

4.1.1 a64strcpy

The detailed step by step implementation for a64strcpy is given below. We take an example where the source address is byte aligned and destination address is halfword aligned. l0-l1, l2-l3 and o2-o3 are register pairs used in the program, 'b' refers to byte sized data, 'h' refers to halfword sized data and 'dw' refers to doubleword data. Figure 4.1 shows the string alignments at the the start of the program.

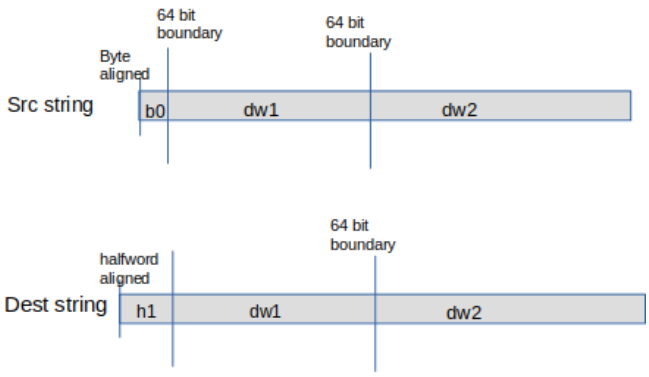


Figure 4.1: source and destination alignments

- STEP 1 - Initial string alignment is given in figure 4.1. Store byte b0 from source string to destination string at h1. Now, source is doubleword aligned and destination is byte aligned as b0 will only fill half of h1.
- STEP 2 - Load a new doubleword from source in register pair, as seen in figure 4.2. Load b1 into destination. This makes the destination doubleword aligned.

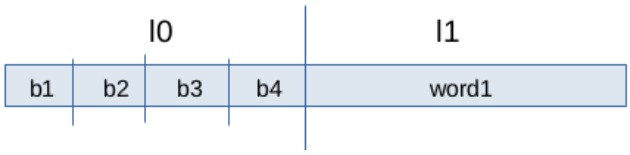


Figure 4.2: newly loaded doubleword

- STEP 3 - Shift l0-l1 left by 8 bits and store in l2-l3 as seen in figure 4.3. As we have stored b1 already, we do not need it.

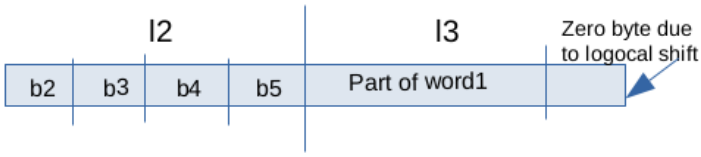


Figure 4.3: intermediate doubleword

- STEP 4 - Load a new doubleword into l0-l1. We need the most significant byte from the new doublword. Hence we shift l0-l1 right by 7 bytes and save it in another register pair, o2-o3, as seen in figure 4.4.

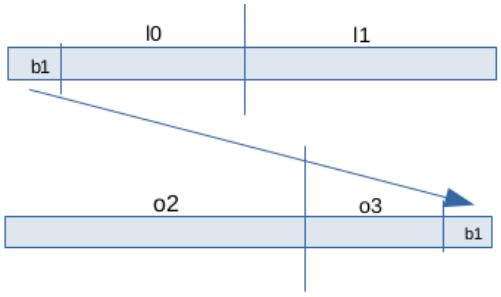


Figure 4.4: shifting of l0-l1 to get required byte

- STEP 5 - Now we OR the two registers – l2-l3 and o2-o3. We now have a doubleword we can store in the destination.
- STEP 6 - Store the doubleword and and repeat from step2 till we see null byte.

After loading each new byte/doubleword from source, it is checked for null byte. Only if null byte is not found does the program flow continue as given above. Else, we check the doubleword bytewise for the null byte and store only till the null byte is found.

4.1.2 a64strcat

a64strcat differs from a64strcpy in the location of where the source string needs to be copied into. a64strcpy copies into the start of the location of destination string, while a64strcat copies where the destination string terminates. Hence, in a64strcat we first calculate the destination address where the string is to be copied. The destination address is calculated by looking for the null byte, done bytewise till 64 bit alignment is found and then done in 64 bit data chunks. We then follow the exact flow of a64strcpy.

4.1.3 a64strncpy

In a64strncpy we have to copy for n bytes, where n is given by the user. Hence, we have the added condition of checking if required bytes have been copied.

Hence after loading, checking for null byte and manipulating the source doubleword, and before performing a store we check value of n, if n is less than 8, we store byte by byte till n is 0. Else we continue like in a64strcpy and will add null bytes after the source string has been completely copied, till n is 0.

4.1.4 a64strcmp

In a64strcmp we have to load both the strings and do a compare.

- As given in in figure 4.1, assume that source is byte aligned and destination is doubleword aligned.
- Similar to step 1 of the a64strcpy, first align source string to the double word boundary by performing a bytewise compare of b0 and part of h1 (see figure 4.1). Now only destination is unaligned to a byte boundary.
- Now, load a new doubleword from the source string, and compare its first byte b1 with the remaining byte of h1 of destination string.

- Now, similar to step 3 of the a64strcpy example, shift the remainder of the source string by 8 bits, to save the remaining part of the doubleword that is yet to be compared to.
- Again load a new source doubleword, shift it and OR it with the previously saved source string, such that its most significant byte is concatenated to the previously saved source string, as seen in figure 4.4.
- Compare this doubleword with the destination string in one cycle.

4.1.5 a64strcasecmp

a64strcasecmp ignores the case of the character while performing the compare. We have to make the string to be compared lowercase before we perform the compare. Hence, this has same program flow as a64strcmp, except that each time new data is loaded (of either of the strings), we convert it to lowercase. This conversion is done by oring each byte with 0x20.

4.1.6 a64strlen

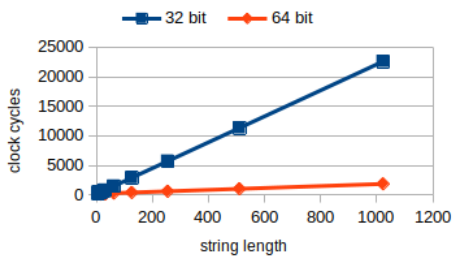
As this function uses a single string we first load the data bytewise till doubleword alignment is reached, while increasing count by 1. Then we load data in 64 bit chunks, and increment count by 8, till null byte is found.

Chapter 5

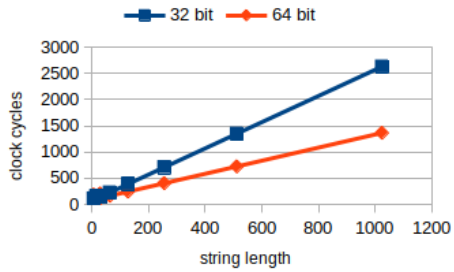
Performance characterization

We have compared the string libraries with the existing uclibc C string functions.

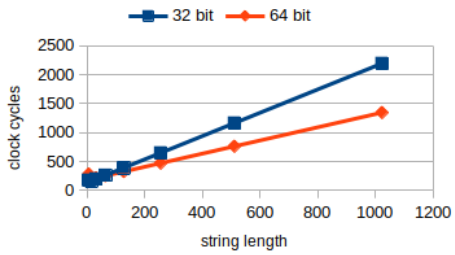
For string libraries, functions are compared at various string lengths. Lengths are as 8,16, 32...1024 bytes. We plot the clock cycles required by the the 32 bit and 64 bit libraries as function of the string length. In each case improvement is noted.



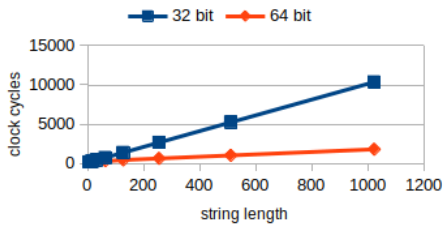
(a) a64strcasecmp vs strcasecmp



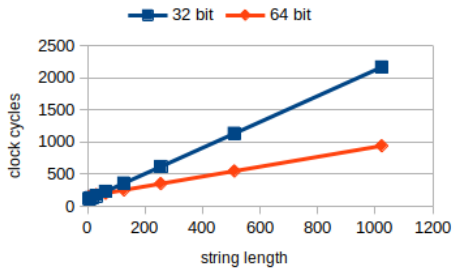
(b) a64strcmp vs strcmp



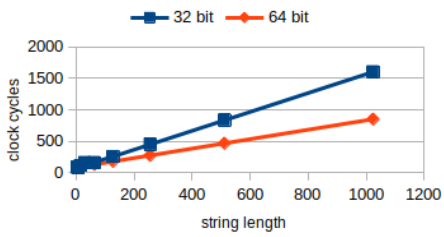
(c) a64strcat vs strcat



(d) a64strncpy vs strncpy



(e) a64strcpy vs strcpy



(f) a64strlen vs strlen

Figure 5.1: String function performance comparison

The performance improvement is summarized in table 5.1. Improvement is given in terms of ratio of 32 bit clock cycles to 64 bit clock cycles for a string length of 1024 bytes.

function	improvement
a64strcmp	1.92
a64strcasecmp	12.21
a64strcpy	2.3
a64strncpy	5.73
a64strcat	1.63
a64strlen	1.88

Table 5.1: Performance improvement of 64 bit vs 32 bit string functions in terms of required clock cycle ratio at 1024 bytes

We can see significant performance improvement for all libraries. As can be seen from the graphs, the performance gets better as the string length increases. For most functions, we can start to see improvement after a length of 32 bytes. As seen above, many applications have data lengths more than twice of 32, hence this library can be used for various applications.

Chapter 6

Verification of libraries

The verification of string libraries has three parts -

- Lengths - The tests need to cover a wide range of lengths.
- Alignments - The tests should cover all possible alignment combinations of source and destination string.. Both source and destination strings be doubleword, word, halfword or byte aligned. As there are 8 bytes in a doubleowrd, their are 8 cases of alignments for each. Hence a minimum of 49 combinations need to be tested.
- Corner cases - Output should be correct for all possible outcomes of the functions. Tests for -
  - a64strcmp and a64strcasecmp - Give correct output when strings are equal or unequal. Output should be zero if equal, else, the difference of the unequal byte of the two strings.
  - a64strncpy - correct output to be checked when n is less than length of string and n is greater than length of string. In both cases only n bytes should be copied.
  - a64strcpy and a64strcat - detect the null byte and copy/concatenate string correctly.
  - a64strlen - output should be correct for all lengths.

We will first start with by initiating a for loop for the string lengths. It will start with 8,16...2048. We will then initiate another loop for all alignments. A nested loop is created, one outer loop for destination alignments and inner loop for source alignments. Hence for each alignment of the destination all source alignments are tested. The loop increments the string address that is passed to the function. For e.g., we first pass a doubleword aligned string of length 8, then in the next iteration we increment the address by 1, hence we pass a byte aligned string of length 7.

This also reduces the length of the string. Hence, for e.g., when the starting length is 8, lengths of 7 to 0 are tested too.

We loop it 8 times as there are 8 cases of alignments – starting with doubleword aligned, 1 position where the string is word aligned , 2 positions where string is halfword aligned and 4 positions where it is byte aligned. Hence this test will cover 49 possible combinations. It also tests for all possible locations of null byte.

To check for all corner cases, individual tests are written for each case.

Chapter 7

Conclusion

High performance string libraries were developed using the 64 bit vector instructions in the AJIT processor. All functions showed significant improvement in performance as compared to their 32 bit non-vectored counterparts. Performance improvement of up to 91% is observed.

References

[1] *System V Application Binary Interface.* SPARC International Inc., third edition, 1990.

[2] *The SPARC Architecture Manual.* SPARC International Inc., Menlo Park, CA 94025, 1991.

[3] Silva T. Almeida T., Hidalgo J. Towards sms spam filtering: Results under a new dataset. *INTERNATIONAL JOURNAL OF INFORMATION SECURITY SCIENCE*, 2013.

[4] guineyhttps://stackoverflow.com/users/214325/hugh-guiney. Typical url lengths for storage calculation purposes (url-shortener).

[5] SALT DATA LABS. Bert: How to handle long documents.

[6] Desai M. *Generic Core User Guide for AJIT Processor.* IIT Bombay.